

GitHub mit Claude

Was GitHub ist, was du dort alles nutzen kannst, und wie Claude direkt in deinem Repository mitarbeitet. Mit Skizze und einem Beispiel aus deinem eigenen Projekt.

Stand
28.09.2026
Repository
...-ship-it/Lukas

TEIL A

GitHub verstehen

01 Was ist GitHub?

GitHub ist eine Online-Plattform, auf der du Projekte speicherst, jede Änderung nachverfolgst und mit anderen (auch mit Claude) daran arbeitest. Ein Projekt heißt dort *Repository*, kurz Repo. Es ist im Kern ein Ordner mit Dateien, der sich an jede frühere Fassung erinnert.

Man kann es sich wie einen Cloud-Ordner vorstellen, nur mit drei Unterschieden:

- **Lückenloser Verlauf.** Jede Änderung wird als *Commit* gespeichert: wer hat was, wann und warum geändert. Du kannst jederzeit zu einem alten Stand zurück.
- **Gefahrloses Ausprobieren.** Auf einem *Branch* (Nebenzweig) wird gearbeitet, ohne die gültige Fassung auf `main` anzufassen.
- **Prüfen vor Übernehmen.** Ein *Pull Request* zeigt dir jede geänderte Zeile. Erst wenn du zustimmst, wandert die Änderung in `main`.

GIT ODER GITHUB?

Git ist das Programm, das Versionen verwaltet (läuft auf jedem Rechner). **GitHub** ist die Website, auf der Git-Projekte online liegen, dazu Aufgabenlisten, Automatisierung und Webseiten-Hosting. Claude nutzt beides: Git für Commits, GitHub für Branches, Pull Requests und Issues.

Dein Repo `lukasfabiogrosser-ship-it/Lukas` ist genau so ein Ordner. Darin liegen heute deine Handbücher (`handbuch/`) und das Archiv (`archiv/`), mit drei Commits Verlauf.

02 Die Skizze: Was du auf GitHub nutzen kannst

In der Mitte steht dein Repository. Alles andere hängt daran. Claude (unten rechts, blau) kann direkt im Repo lesen, schreiben und Änderungen einreichen.

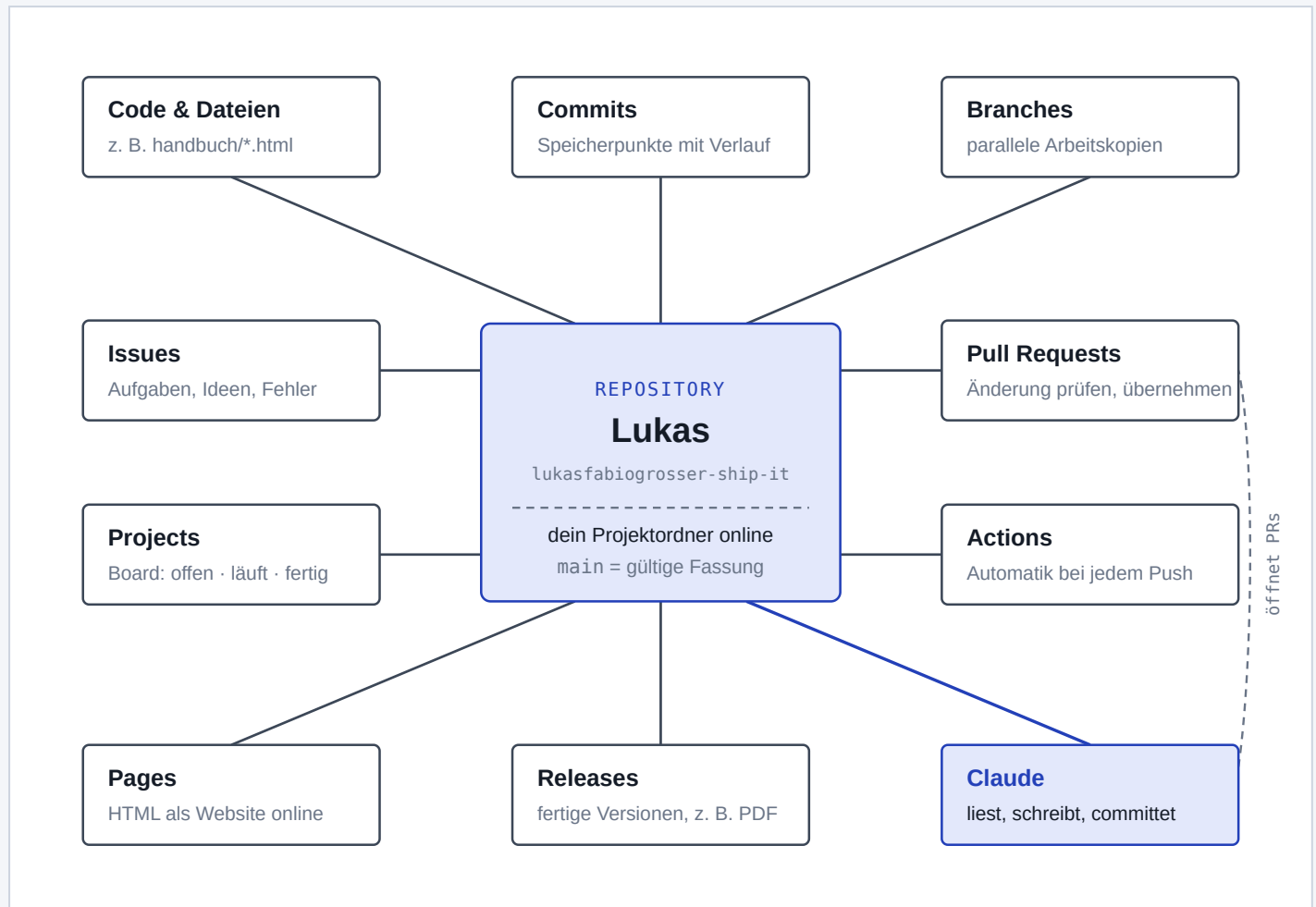


Abb. 1 Die zehn Bausteine rund um ein Repository. Die gestrichelte Linie rechts: Claude reicht seine Arbeit als Pull Request ein, du entscheidest.

03 Die Bausteine im Einzelnen

Für den Anfang brauchst du nur die ersten fünf. Der Rest kommt, wenn dein Projekt wächst.

BAUSTEIN	WAS ES IST	FÜR DICH KONKRET
Repository	Projektordner mit Verlauf	Lukas mit Handbüchern und Archiv
Commit	Ein gespeicherter Änderungsschritt mit Nachricht	„Add handbook volume 2 on data, memory ...“
Branch	Nebenzweig zum Arbeiten, main bleibt unberührt	Claude arbeitet gerade auf <code>claude/wonderful-newton-q3lrid</code>
Pull Request	Antrag, einen Branch in main zu übernehmen, mit Zeilenvergleich	Neue Seite prüfen, dann „Merge“ klicken
Issue	Aufgabe oder Idee mit Diskussion	„Band 4 zu Steuern für Trader schreiben“
Projects	Kanban-Board aus Issues	Spalten: Idee · in Arbeit · veröffentlicht
Actions	Automatische Abläufe bei Ereignissen	Bei @claude im Issue startet Claude von selbst
Pages	Kostenloses Hosting für HTML-Dateien	Deine Handbücher als echte Website unter <code>...github.io/Lukas/</code>
Releases	Benannte Versionen mit Downloads	„Handbuch v2.0“ mit den PDF-Dateien

HINWEIS ZU PAGES

GitHub Pages ist für öffentliche Repositories kostenlos. Für private Repositories brauchst du einen bezahlten GitHub-Plan. Öffentlich heißt: jeder kann die Dateien lesen.

TEIL B

Claude mit GitHub nutzen

04 Drei Wege, Claude mit GitHub zu verbinden

WEG	SO FUNKTIONIERT ES	GUT FÜR
Claude Code im Web / in der App claude.ai/code	GitHub-Konto verbinden, Repo auswählen, Auftrag schreiben. Claude arbeitet in einer Cloud-Umgebung, committet auf einen eigenen Branch und pusht.	Alles ohne eigenen PC, auch vom Handy. So entsteht gerade diese Seite.
Claude Code im Terminal auf deinem Rechner	Repo mit <code>git clone</code> herunterladen, im Ordner <code>claude</code> starten und Aufträge in normaler Sprache geben.	Längere Arbeit am eigenen Rechner, Zugriff auf lokale Dateien.
@claude in GitHub GitHub App + Action	Einmal einrichten, dann in einem Issue oder Pull Request <code>@claude</code> erwähnen. Claude antwortet dort und kann Änderungen einreichen.	Aufgaben direkt auf GitHub verteilen, ohne eine Sitzung zu öffnen.

Außerdem kannst du im normalen Claude-Chat (claude.ai) GitHub als Quelle anbinden. Dann liest Claude Dateien aus deinem Repo als Kontext, ändert dort aber nichts.

Der Ablauf als Skizze

Egal welcher Weg: Claude arbeitet nie direkt auf `main`, sondern auf einem Branch. Du siehst jede Änderung, bevor sie gilt.

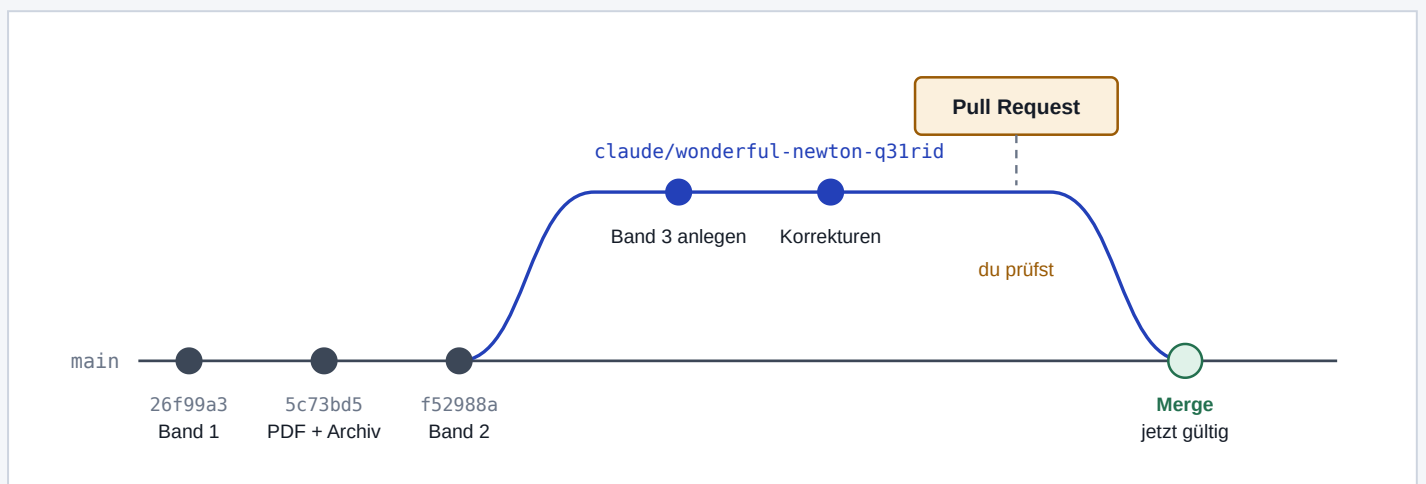


Abb. 2 Die drei grauen Punkte sind die echten Commits in deinem Repo. Der blaue Zweig ist Claudes Arbeitsbereich. Erst mit dem Merge landet die Arbeit in `main`.

Das Beispiel ist genau das, was in dieser Sitzung passiert ist. Du kannst es jederzeit mit einem anderen Thema wiederholen.

1 Repo verbinden. Auf `claude.ai/code` dein GitHub-Konto verknüpfen und das Repository `Lukas` auswählen.

2 Auftrag geben. In normaler Sprache, zum Beispiel: „*Erkläre mir, was GitHub ist, und mach eine Skizze dazu.*“

3 Claude liest das Repo. Es sieht deine Handbücher, übernimmt Farben, Schriften und Aufbau, damit Band 3 zu Band 1 und 2 passt.

4 Claude arbeitet auf einem Branch. Die neue Datei `handbuch/github-mit-claude.html` entsteht auf `claude/wonderful-newton-q31rid`. `main` bleibt unverändert.

5 Commit und Push. Claude speichert die Änderung mit einer Nachricht und lädt sie zu GitHub hoch.

6 Du prüfst. Auf GitHub unter *Pull requests* → *New* den Branch auswählen, den Zeilenvergleich ansehen. Oder Claude bitten: „*Erstelle einen Pull Request.*“

7 Übernehmen. Auf „Merge pull request“ klicken. Ab jetzt ist Band 3 Teil der gültigen Fassung.

WARUM DAS SICHER IST

Bis Schritt 7 kannst du alles verwerfen. Selbst nach dem Merge bleibt jeder alte Stand im Verlauf erhalten und lässt sich wiederherstellen.

@claude direkt in Issues und Pull Requests

Mit der Claude-GitHub-App kannst du Claude Aufgaben geben, ohne eine Sitzung zu öffnen. Am einfachsten richtest du es aus Claude Code im Terminal ein: den Befehl `/install-github-app` eingeben und den Schritten folgen. Er installiert die App und legt den nötigen Schlüssel als geheimes Repository-Secret ab.

Danach liegt im Repo eine Datei wie diese:

.GITHUB/WORKFLOWS/CLAUDE.YML · BEISPIEL

```
name: Claude
on:
  issue_comment:
    types: [created]
  issues:
    types: [opened]
  pull_request_review_comment:
    types: [created]

jobs:
  claude:
    if: contains(github.event.comment.body, '@claude') ||
contains(github.event.issue.body, '@claude')
    runs-on: ubuntu-latest
    permissions:
      contents: write
      issues: write
      pull-requests: write
      id-token: write
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
```

Ein Issue sieht dann so aus:

ISSUE #1 · VON DIR

Glossar für alle Bände

@claude sammle alle Fachbegriffe aus
handbuch/ und lege eine Seite
handbuch/glossar.html im gleichen Stil an.

ANTWORT · CLAUDE (BEISPIEL)

Ich habe 48 Begriffe aus beiden Bänden
gesammelt und die Seite angelegt. Die
Änderung liegt auf einem eigenen Branch, der
Pull Request ist verlinkt.

KOSTEN

Die Action läuft über GitHub-Actions-Minuten und über die Claude-API bzw. dein Claude-Abo. Für ein kleines Repo wie deins bleibt das überschaubar, sollte aber einmal im Blick behalten werden.

Aufträge zum Kopieren

Diese Sätze funktionieren in Claude Code im Web, im Terminal und nach `@claude` in einem Issue.

ÜBERBLICK · WAS STECKT IM REPO?

Lies das Repository und erkläre mir in einfachen Worten, welche Dateien es gibt, wofür sie da sind und was man als Nächstes verbessern könnte.

NEUER INHALT · IM BESTEHENDEN STIL

Schreibe ein neues Kapitel über [Thema] für mein Handbuch. Übernimm Aufbau, Farben und Schriften aus `handbuch/claude-aufruesten.html`. Committe auf deinem Branch und erstelle danach einen Pull Request, damit ich es prüfen kann.

WEBSITE · GITHUB PAGES EINRICHTEN

Bereite das Repo für GitHub Pages vor: Lege eine `index.html` als Startseite an, die auf alle Handbücher und PDFs verlinkt. Sag mir danach, welche Einstellung ich auf GitHub unter Settings → Pages setzen muss.

AUFRÄUMEN · VERLAUF VERSTEHEN

Zeig mir die letzten Commits und fasse für jeden in einem Satz zusammen, was sich geändert hat.

Fünf Regeln und ein kleines Glossar



Nie Passwörter committen

API-Schlüssel gehören in Settings → Secrets, nie in eine Datei im Repo.



Erst prüfen, dann mergen

Jeder Pull Request zeigt die Änderungen Zeile für Zeile.



main schützen

Unter Settings → Branches festlegen, dass nur per Pull Request geändert wird.



Klare Aufträge

Ziel, Datei und gewünschtes Ergebnis nennen. Je genauer, desto besser.



Privat oder öffentlich?

Öffentliche Repos kann jeder lesen.
Handelsdaten bleiben privat.

clone	Ein Repo von GitHub auf den eigenen Rechner kopieren.
push	Lokale Commits zu GitHub hochladen.
pull	Neue Commits von GitHub herunterladen.
merge	Einen Branch in einen anderen übernehmen, meist in <code>main</code> .
fork	Eine eigene Kopie eines fremden Repos anlegen.
diff	Die Gegenüberstellung von altem und neuem Stand einer Datei.
CLAUDE.md	Eine Datei im Repo, die Claude bei jeder Sitzung zuerst liest: deine Ziele, Regeln und Vorlieben.

Praxis-Handbuch Band 3 · Stand 28.09.2026. Anleitungen zu Claude Code und zur
GitHub-Integration: code.claude.com/docs. Hilfe zu GitHub selbst: docs.github.com.